

Multiplicación rápida de enteros grandes: el algoritmo de Karatsuba

Informe de la Práctica N.º 2

Emmanuel Castaño Sepúlveda
Luis Miguel Mira Mejia

Escuela de Ciencias Aplicadas e Ingeniería, Universidad EAFIT
Análisis y Diseño de Algoritmos
Profesor Carlos Alberto Alvarez Henao
1 de noviembre de 2026

1. Deducción de la identidad de Karatsuba

Se parten x y y (de n dígitos) en una mitad alta y una baja de $m = \lfloor n/2 \rfloor$ dígitos:

$$x = x_1 \cdot 10^m + x_0, \qquad y = y_1 \cdot 10^m + y_0.$$

Al multiplicar aparecen **cuatro** productos de la mitad del tamaño:

$$xy = x_1 y_1 \cdot 10^{2m} + (x_1 y_0 + x_0 y_1) \cdot 10^m + x_0 y_0.$$

El término central se puede obtener con un solo producto, porque

$$(x_1 + x_0)(y_1 + y_0) = x_1 y_1 + (x_1 y_0 + x_0 y_1) + x_0 y_0.$$

Con $P_1 = x_1 y_1$, $P_2 = x_0 y_0$ y $P_3 = (x_1 + x_0)(y_1 + y_0)$, el término central es $P_3 - P_1 - P_2$, y queda

$$\boxed{xy = P_1 \cdot 10^{2m} + (P_3 - P_1 - P_2) \cdot 10^m + P_2}$$

Son **tres** productos en lugar de cuatro. A cambio se hacen dos sumas y dos restas, que cuestan solo $\Theta(n)$.

Ejemplo. $1234 \cdot 5678$, con $m = 2$: $P_1 = 12 \cdot 56 = 672$, $P_2 = 34 \cdot 78 = 2652$ y $P_3 = 46 \cdot 134 = 6164$. El término central es $6164 - 672 - 2652 = 2840$, y $672 \cdot 10^4 + 2840 \cdot 10^2 + 2652 = 7\,006\,652$, que es el producto correcto.

Detalles de implementación.

- Se resta en el orden $(P_3 - P_1) - P_2$. Como $P_3 - P_1 = P_2 + x_1 y_0 + x_0 y_1 \geq P_2$, nunca aparece un resultado negativo.
- Los operandos se rellenan con ceros hasta una potencia de 2 una sola vez, al inicio. Si se rellenara en cada nivel, la rama de P_3 (que puede tener $m+1$ dígitos) volvería al tamaño n y la recursión no avanzaría.

2. Resolución de la recurrencia

Cada llamada hace 3 llamadas sobre la mitad del tamaño y un trabajo lineal (partir, sumar, restar y desplazar):

$$T(n) = 3\,T(n/2) + cn, \qquad T(1) = d.$$

2.1. Teorema Maestro

- Parámetros: $a = 3$, $b = 2$, $f(n) = \Theta(n)$.
- Comparación: $n^{\log_b a} = n^{\log_2 3} \approx n^{1{,}585}$, que crece más rápido que $f(n) = n$.
- Caso 1, porque $f(n) = O(n^{\log_2 3 - \varepsilon})$ con $\varepsilon \approx 0{,}585 > 0$. Por tanto, $T(n) = \Theta(n^{\log_2 3})$.

Con cuatro productos sería $a = 4$ y $T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$, igual que el método escolar. Toda la ganancia viene de pasar de 4 a 3 productos.

2.2. Árbol de recurrencia

Nivel	Subproblemas	Tamaño	Costo del nivel
0	1	n	cn
1	3	$n/2$	$\frac{3}{2}\,cn$
i	3^i	$n/2^i$	$\left(\frac{3}{2}\right)^i cn$
$\log_2 n$ (hojas)	$n^{\log_2 3}$	1	$d\,n^{\log_2 3}$

Tabla 1. Árbol de recurrencia de $T(n) = 3T(n/2) + cn$.

- **Altura:** $\log_2 n$, porque el tamaño se divide entre 2 hasta llegar a 1 dígito.
- **Costo por nivel:** crece en un factor 3/2, porque los subproblemas se triplican y su tamaño solo se reduce a la mitad.
- **Suma total:** es una serie geométrica creciente, así que el costo se concentra en los niveles profundos:

$$T(n) = \sum_{i=0}^{\log_2 n - 1} \left(\tfrac{3}{2}\right)^i cn + d\,n^{\log_2 3} = (d + 2c)\,n^{\log_2 3} - 2cn = \Theta\Big(n^{\log_2 3}\Big).$$

El árbol aporta algo que el teorema no muestra: el término $-2cn$. Por él, en tamaños finitos la pendiente medida queda un poco por encima de 1,585 (sección 3).

2.3. Por qué el exponente es $\log_2 3$

Cada vez que el tamaño se divide entre 2, los subproblemas se multiplican por 3. Tras $\log_2 n$ divisiones hay $3^{\log_2 n} = n^{\log_2 3}$ hojas. Con 4 productos el exponente sería $\log_2 4 = 2$; con 3 es $\log_2 3 \approx 1{,}585$.

3. Análisis experimental

3.1. Montaje

- Números como cadenas de dígitos en base 10 (`std::string`), sin tipos nativos para el producto completo.
- Tamaños de 2 a 4096 dígitos (potencias de 2), operandos aleatorios con semilla fija.
- `steady_clock` (resolución de 21 ns), lotes de al menos 50 ms y mediana de 5 lotes por algoritmo.
- Cada resultado se acumula en una suma de control y se verifica, fuera del cronómetro, que ambos algoritmos coincidan.
- `g++` 11.4 con `-O2`, Ubuntu 22.04 (WSL2), AMD Ryzen 7 7730U conectado a la corriente.
- Serie principal con caso base de 1 dígito (umbral 1) y serie complementaria con umbrales de 16, 32 y 64 dígitos.

3.2. Tabla de tiempos

	Tiempo (μs)		Razón Karatsuba / clásica			
n	Clásica	Karatsuba	Umbral 1	Umbral 16	Umbral 32	Umbral 64
2	0,098	0,480	4,91	1,71	1,77	1,74
8	0,374	12,4	33,20	1,20	1,21	1,25
32	4,19	107	25,62	1,57	1,03	1,05
64	16,6	324	19,53	1,43	1,03	1,00
128	66,8	972	14,55	1,12	**0,84**	**0,83**
256	265	2.948	11,14	**0,92**	0,69	0,64
512	1.066	9.040	8,48	0,73	0,55	0,51
1024	4.258	27.400	6,44	0,58	0,44	0,41
2048	16.903	84.871	5,02	0,44	0,33	0,31
4096	68.578	247.099	3,60	0,33	0,26	0,24

Tabla 2. Tiempo por multiplicación con caso base de 1 dígito, y razón de tiempos por umbral. Razón menor que 1: Karatsuba es más rápida; en negrita, el primer tamaño en que ocurre. Los datos completos están en el repositorio.

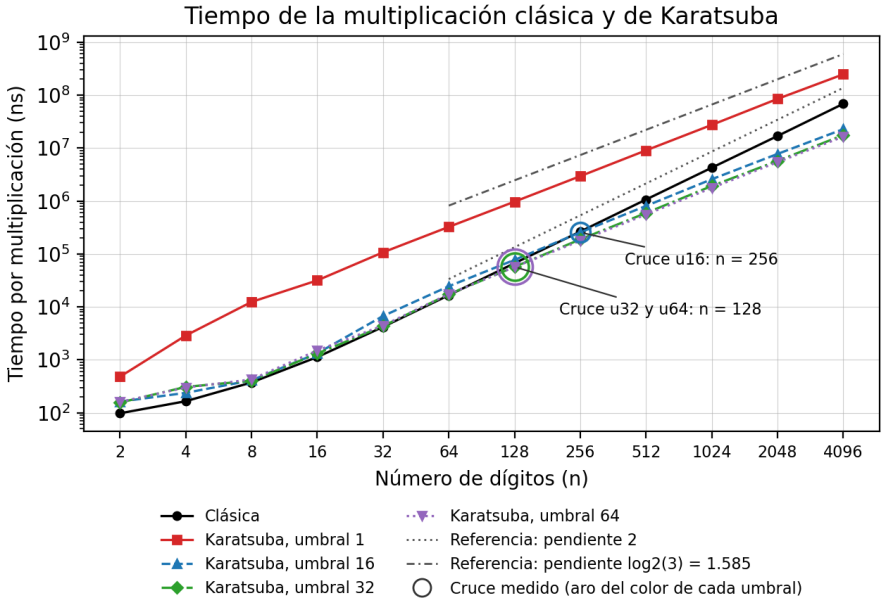


Figura 1. Tiempo por multiplicación en escala log-log, con rectas de referencia de pendiente 2 y $\log_2 3$.

Exponentes medidos. En escala log-log, la pendiente de cada curva es su exponente. Desde $n = 256$ se obtuvo 2,002 para la clásica (teórico 2) y 1,601 para Karatsuba (teórico 1,585). Contando solo los productos de un dígito, Karatsuba alcanza exactamente 1,585: el algoritmo hace el trabajo que predice la recurrencia.

3.3. Punto de cruce

- **Umbral 1:** no hay cruce hasta 4096 dígitos (Karatsuba es 3,6 veces más lenta), aunque la diferencia se reduce en cada duplicación. Extrapolando las dos curvas, el cruce estaría cerca de 10^5 dígitos.

- **Umbrales 16, 32 y 64:** el cruce aparece en $n = 256$, 128 y 128. En 4096 dígitos, Karatsuba con umbral 32 o 64 tarda cerca de la cuarta parte que la clásica.

3.4. Por qué la clásica gana en números pequeños

La complejidad asintótica solo compara exponentes; el tiempo real también depende de las constantes ocultas. Si $t_c = c_c\, n^2$ y $t_k = c_k\, n^{\log_2 3}$, el cruce ocurre en $n^* \approx (c_k/c_c)^{2{,}41}$: si la constante de Karatsuba se duplica, el cruce se aleja unas 5 veces.

- **Copias:** en cada llamada, `partir`, `sumar`, `restar` y `desplazar` crean cadenas nuevas.
- **Asignaciones de memoria:** cada cadena nueva reserva y libera memoria. La clásica reserva un solo vector y hace un ciclo doble.
- **Resultado:** con 8 dígitos, Karatsuba crea unas 40 llamadas recursivas para evitar parte de 64 productos simples, y es 33 veces más lenta.
- **Solución:** usar la clásica por debajo de un umbral elimina los niveles con más llamadas. Con 32 a 64 dígitos el cruce baja a 128, sin cambiar el exponente.

4. Conclusiones y dificultades encontradas

Conclusiones.

- La identidad de Karatsuba reemplaza un producto por sumas y restas, y lleva el costo de $\Theta(n^2)$ a $\Theta(n^{\log_2 3}) \approx \Theta(n^{1{,}585})$.
- Los experimentos confirman los exponentes: 2,002 para la clásica y 1,601 para Karatsuba.
- Mejor complejidad no significa menor tiempo en tamaños pequeños. Con caso base de 1 dígito, las copias y asignaciones de memoria dejan el cruce cerca de 10^5 dígitos.
- Un caso base de 32 a 64 dígitos hace a Karatsuba más rápida desde 128 dígitos.

Dificultades.

- Rellenar en cada nivel impedía que la recursión avanzara; se rellenó una sola vez.
- Para evitar restas negativas se fijó el orden $(P_3 - P_1) - P_2$.
- Los tiempos pequeños estaban cerca de la resolución del reloj; se midió en lotes de 50 ms.
- Dos corridas se afectaron por la suspensión del equipo y por el modo de energía; se repitieron en las mismas condiciones.

Declaración de uso de inteligencia artificial

Se usaron herramientas de IA como apoyo. Las decisiones de diseño, la verificación de resultados y los commits fueron de los integrantes, y ambos revisamos el código y el informe.

- **Luis:** OpenAI Codex, para proponer la estructura del código, implementar y probar la multiplicación clásica y las funciones auxiliares, y redactar su documentación. Python se usó solo como referencia externa para validar resultados.
- **Emmanuel:** Claude (claude.ai), para planear el trabajo por fases y como apoyo en la redacción del informe, cuyo contenido y cifras revisamos ambos; y Claude Code, para implementar, probar, medir y analizar Karatsuba, con revisión de cada fase antes de aceptarla.

El detalle está en `docs/uso_ia_luis.md` y `docs/uso_ia_emmanuel.md`.